Robust KalmaNet State Estimation for QCar

Physics Prediction + Learned Robust GPS/Velocity Update

Quang Huy Nugyen

Multi-Vehicle Self-Driving Real QCar Project

May 1, 2026

RKNet estimates $\boldsymbol{x} = [x, y, \psi, v]^T$ while rejecting corrupted GPS and velocity measurements.

Problem Setup: Signals and Measurement

Measurement vector

$$\boldsymbol{z}_k = \begin{bmatrix} x_{gps,k} & y_{gps,k} & \psi_{fused,k} & v_{tach,k} \end{bmatrix}^T$$

Raw branch inputs

- IMU: $[v, \psi, a_x, a_y, \omega_z]$
- Steering: $[v, \psi, \delta]$
- Wheel: $[v, \psi, v_{fl}, v_{fr}, v_{rl}, v_{rr}]$

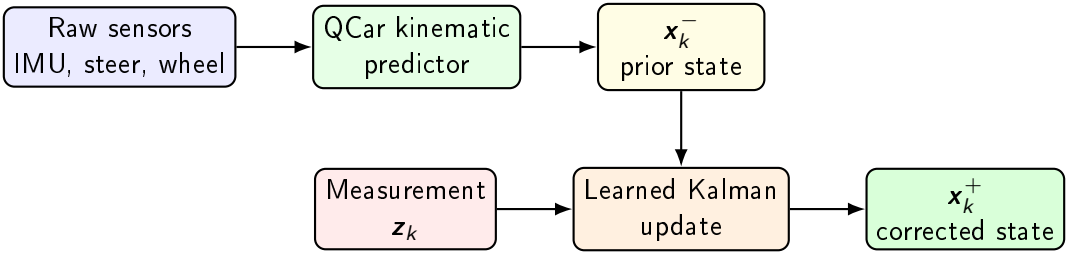
Availability information

- GPS position valid flag
- Heading valid flag
- GPS hold valid flag
- GPS age in seconds

Observation model

$$\boldsymbol{z}_k = H\boldsymbol{x}_k + \boldsymbol{n}_k, \qquad H \approx I_4$$

RKNet Concept Design



- Current configuration uses `predictor_mode: kinematic`.
- Prediction is physics-based, not learned.
- Update learns measurement reliability mask and Kalman gain.
- Main robustness target: GPS position and velocity corruption.

Prediction Math: QCar Bicycle Model

Yaw-rate and local motion

$$\dot{\psi}_k = \frac{v_{pose,k}\tan(\delta_k)}{L}, \qquad \Delta x_E = v_{pose,k}\Delta t, \qquad \Delta y_E = 0$$

Global prediction

$$x_k^- = x_{k-1}^+ + v_{pose,k}\cos(\psi_{k-1}^+)\Delta t$$

$$y_k^- = y_{k-1}^+ + v_{pose,k}\sin(\psi_{k-1}^+)\Delta t$$

$$\psi_k^- = \mathrm{wrap}\left(\psi_{k-1}^+ + \frac{v_{pose,k}\tan(\delta_k)}{L}\Delta t\right)$$

Velocity lag model used by current config

$$a_k = \frac{-v_{k-1}^+ + g u_k}{\tau}, \qquad v_k^- = \mathrm{clip}\left(v_{k-1}^+ + a_k\Delta t\right)$$

Config values: $L = 0.2$, $\tau = 0.301$, $g = 6.598$, $|v| \le 2.0$.

Learned Robust Update Math

Innovation

$$\boldsymbol{r}_k = \mathrm{wrap}(\boldsymbol{z}_k - H\boldsymbol{x}_k^-), \qquad \Delta\boldsymbol{z}_k = \mathrm{wrap}(\boldsymbol{z}_k - \boldsymbol{z}_{k-1})$$

Updater features

$$\Delta\boldsymbol{x}_k = \mathrm{wrap}(\boldsymbol{x}_k^- - \boldsymbol{x}_{k-1}^+)$$

$$\bar{\boldsymbol{r}}_k = \boldsymbol{a}_k \odot \boldsymbol{r}_k, \qquad \bar{\Delta\boldsymbol{z}}_k = \boldsymbol{a}_k \odot \Delta\boldsymbol{z}_k$$

$$\eta_k = \log(1 + \min(age_k, 10))$$

$$\boldsymbol{f}_k = \left[\frac{\Delta\boldsymbol{x}_k}{\boldsymbol{s}_x}, \frac{\bar{\boldsymbol{r}}_k}{\boldsymbol{s}_z}, \frac{\bar{\Delta\boldsymbol{z}}_k}{\boldsymbol{s}_z}, \boldsymbol{a}_k, g_k^{hold}, \eta_k, \log(1 + |\bar{\boldsymbol{r}}_k|), \log(1 + |\bar{\Delta\boldsymbol{z}}_k|)\right]$$

For $[x, y, \psi, v]$, $\boldsymbol{a}_k = [g_k, g_k, h_k, 1]^T$ and $\boldsymbol{f}_k \in \mathbb{R}^{26}$.

Measurement mask

$$\boldsymbol{m}_k^z = \sigma(MLP(\boldsymbol{f}_k))$$

$$\tilde{\boldsymbol{r}}_k = \boldsymbol{m}_k^z \odot \boldsymbol{a}_k \odot \boldsymbol{r}_k$$

Learned Kalman gain

$$\boldsymbol{h}_k = GRU(\boldsymbol{m}_k \odot \boldsymbol{f}_k, \boldsymbol{h}_{k-1})$$

$$K_k = s\tanh(MLP(\boldsymbol{h}_k))$$

Final update

$$\boldsymbol{x}_k^+ = \boldsymbol{x}_k^- + K_k\tilde{\boldsymbol{r}}_k$$

Attack Setup: Focus on GPS and Velocity

GPS position attacks

- Noise: $z_{xy}^{atk} = z_{xy} + \epsilon$
- Jump: $z_{xy}^{atk} = z_{xy} + \boldsymbol{b}_{jump}$
- Freeze: $z_{xy,k}^{atk} = z_{xy,k_0}$
- Dropout: $gps\_valid_k = 0$
- Reacquisition offset

Velocity attacks

- Bias: $z_v^{atk} = z_v + b_v$
- Scale: $z_v^{atk} = \alpha z_v$
- Freeze: $z_{v,k}^{atk} = z_{v,k_0}$
- Noise, ramp, or zero-out
- Wheel branch: $v_{fl}, v_{fr}, v_{rl}, v_{rr}$

Current augmentation probabilities

$$P(GPS) = 0.45, \quad P(\textit{direct meas.}) = 0.35, \quad P(\textit{raw branch}) = 0.25$$

Maximum attacked raw branches per sample: 1.

Expected behavior: $\boldsymbol{m}_{k,i}^z \approx 1$ for clean channels and $\boldsymbol{m}_{k,i}^z \approx 0$ for attacked GPS/velocity channels.

Dataset and Training

Dataset generation

- Recorder saves `datasets/*.npz` and `*.json`.
- Saved fields: raw sensors, GPS flags, z_k, target x_k^*, timestamps.
- Active example: `V0_20260417_094048.npz`

$$\{raw, z, x^*, x_0, \Delta t\} \rightarrow [t : t+T]$$

Training schedule

- $T = 50$, Phase C $T = 80$, stride 1.
- Validation split: 20% tail.
- Phase A: predictor only, skipped for kinematic predictor.
- Phase B: train updater only.
- Phase C: runtime-style fine tuning, teacher forcing off.

Loss: accuracy + robust correction behavior

$$\mathcal{L} = \lambda_u L_{upd} + \lambda_p L_{pred} + \lambda_m L_{mask} + \lambda_K L_{gain} + \lambda_{\Delta K} L_{\Delta K} + \lambda_{\Delta m} L_{\Delta m}$$

$$L_{upd} = MSE_W(x^+, x^*), \qquad L_{pred} = MSE_W(x^-, x^*)$$

$$L_{mask} = BCE(m^z, 1 - \ell^{atk}), \qquad L_{gain} = \|K\|_2^2$$

$$L_{\Delta K} = \|K_k - K_{k-1}\|_2^2, \qquad L_{\Delta m} = \|m_k^z - m_{k-1}^z\|_2^2$$

Meaning

- L_{upd}: final corrected state should match the reference.
- L_{pred}: prior prediction should stay physically reasonable.
- L_{mask}: clean channel target = 1, attacked channel target = 0.

Stability terms

- L_{gain}: avoid very large Kalman gains.
- $L_{\Delta K}$: avoid gain chatter.
- $L_{\Delta m}$: avoid mask flicker.

Testing, Diagnostics, and Result

Evaluation metric

$$RMSE_i = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(\hat{x}_{k,i} - x^*_{k,i}\right)^2}$$

Compared methods

- QCar kinematic reference
- Learned RKNet checkpoint
- Target EKF/reference state

Saved validation after timestamp fix

Method	Mean RMSE
Kinematic reference	0.152
Learned RKNet	0.070

Diagnostics: state error, GPS/velocity mask, masked innovation, and Kalman gain during attack/recovery.

Takeaway: RKNet keeps QCar physics, but learns when to trust GPS and velocity.